

Junior Full Stack Developer – Technical Assessment

Project: Support Ticket Management System

Suggested Duration: 3 Days

1. Assessment Objective

Build a working web-based Support Ticket Management System that demonstrates practical full-stack development skills. The project should cover frontend development, backend REST APIs, MySQL database design, authentication and authorization, API integration, testing, Git/GitHub, debugging, and cloud deployment.

2. Business Scenario

A company needs a simple support portal where customers can raise support tickets and support agents can manage and respond to those tickets. The candidate should develop the complete application from frontend through backend and database, and deploy it so that it can be accessed using a public URL.

3. Technology Requirements

Layer	Required	Acceptable Options
Frontend	React.js	React + JavaScript/TypeScript
Backend	REST APIs	Node.js + Express.js OR Python + FastAPI OR Django
Database	MySQL	MySQL
Authentication	JWT	JWT-based authentication
API Testing	Postman	Postman collection
Testing	Unit/API testing	Jest or PyTest
Version Control	Git/GitHub	GitHub repository
Deployment	Publicly accessible application	Any suitable cloud platform

4. User Roles

Role	Main Capabilities	Restrictions
Customer	Register, login, create tickets, view own tickets, add comments	Cannot view or modify another customer's tickets
Support Agent	Login, view tickets, update status/priority, assign tickets, respond	Cannot perform unauthorized administrative actions

5. Functional Requirements

5.1 Authentication

- Customer and agent login.
- Customer registration.
- Password hashing; passwords must never be stored as plain text.

- JWT-based authentication.
- Protected frontend routes.
- Protected backend APIs.
- Role-based authorization.
- Logout functionality.

5.2 Customer Features

- Customer dashboard showing the customer's tickets.
- Create a new ticket with subject, description and priority.
- View ticket details.
- View ticket comments/responses.
- Add comments to an existing ticket.
- Search and filter tickets.

5.3 Support Agent Features

- Agent dashboard showing ticket statistics.
- View all support tickets.
- Search, filter and sort tickets.
- View ticket details.
- Update ticket status.
- Update ticket priority.
- Assign a ticket to an agent.
- Add responses/comments.

6. Required REST APIs

Method	Endpoint	Purpose	Expected Access
POST	/api/auth/register	Register customer	Public
POST	/api/auth/login	Login	Public
GET	/api/tickets	Get tickets	Authenticated
POST	/api/tickets	Create ticket	Customer
GET	/api/tickets/:id	Get ticket details	Authorized user
PUT	/api/tickets/:id	Update ticket	Agent/authorized user
DELETE	/api/tickets/:id	Delete ticket	As defined by candidate
GET	/api/tickets/:id/comments	Get comments	Authorized user
POST	/api/tickets/:id/comments	Add comment	Authenticated
GET	/api/users	Get users/agents where appropriate	Agent

The candidate may add additional APIs where required. APIs should use appropriate HTTP status codes, validate input, return JSON, and provide meaningful error responses.

7. MySQL Database Requirements

At minimum, create the following related tables:

- users: id, name, email, password_hash, role, created_at
- tickets: id, user_id, subject, description, priority, status, assigned_to, created_at, updated_at
- ticket_comments: id, ticket_id, user_id, comment, created_at

The candidate should demonstrate:

- Primary keys and foreign keys.
- One-to-many relationships.
- CRUD operations.
- JOIN queries.
- Basic indexes for frequently searched fields.
- Basic awareness of query performance.
- Database schema and seed/sample data scripts.

8. Example Database Query Requirement

Write a query that returns all open tickets along with the customer's name and email. The query should demonstrate use of a JOIN and filtering.

9. Frontend Requirements

- React-based application.
- Reusable components.
- Forms with client-side validation.
- API integration using fetch, Axios or equivalent.
- Loading and error states.
- Responsive layout.
- Protected routes based on authentication.
- Clear dashboard and ticket-detail views.

10. Security Requirements

- Password hashing.
- JWT validation.
- Authentication versus authorization must be handled separately.
- Customers must not access another customer's tickets.
- Agent-only APIs must be protected.
- Use parameterized queries/ORM methods to reduce SQL injection risk.
- Configure CORS appropriately.
- Do not commit passwords, database credentials or API secrets to GitHub.
- Use environment variables for secrets and configuration.

11. API Testing

Provide a Postman collection covering at least:

- Successful registration and login.
- Invalid login.
- Create ticket.
- Get tickets.
- Get ticket by ID.

- Update ticket.
- Unauthorized API request.
- Forbidden request for an incorrect role.
- Invalid input.
- Not-found ticket.

12. Automated Testing

Implement a minimum of 5–10 meaningful unit/API tests. Examples:

- Valid login succeeds.
- Invalid password is rejected.
- Ticket creation succeeds.
- Unauthorized user cannot access protected data.
- Customer cannot access another customer's ticket.
- Agent can update ticket status.
- Invalid ticket ID returns the appropriate error.

13. Git & GitHub

The candidate must submit a GitHub repository. The repository should contain meaningful commits rather than a single final upload.

Suggested structure:

```
support-ticket-system/
├── frontend/
├── backend/
├── database/
│   ├── schema.sql
│   └── seed.sql
├── tests/
├── README.md
├── .env.example
└── docker-compose.yml (optional)
```

14. Deployment – Mandatory

The candidate must deploy the working application to a cloud platform. The evaluator must be able to access the application without running it locally.

- Public frontend URL.
- Deployed backend/API URL where applicable.
- Cloud-hosted or otherwise remotely accessible MySQL database.
- Environment variables configured securely.
- README containing setup and deployment instructions.

Submission should include:

- Live application URL
- GitHub repository URL
- Postman collection
- README
- Database schema/seed scripts

15. Optional Enhancements

These are not mandatory and should not be required for a junior candidate:

- Docker / Docker Compose.
- TypeScript.
- Pagination.
- Advanced database indexing/query optimization.
- File attachments.
- Email notification.
- CI/CD pipeline.
- Redis caching.
- Cloud monitoring/logging.

16. Suggested Project Timeline

Day	Suggested Work
Day 1	Project setup, database design, authentication APIs
Day 2	Ticket APIs, MySQL integration, authorization
Day 3	React UI, dashboards and API integration
Day 4	Testing, validation, error handling and polish
Day 5	Deployment, README and final verification

End of Technical Assessment